

Two loan platforms, two team shapes, and the problems we have measured.

WHAT THIS DECK IS FOR

To give you the whole picture in one sitting: **what we run**, **how our teams are cut**, and **what is broken** in logging, testing, Bravo, LORA and the core loan system. Every number here was measured in the last six weeks, and every slide names its source.

HOW TO READ IT

The first half is orientation: the systems, the stacks, and the two team archetypes. The second half is the problem list for testing and logging, then the patterns that cut across all of it.

Sources Two analysis packs: `bravo-analysis/docs` and `lora-workspace/docs/production-findings`. Code in 152 repositories, production telemetry in Datadog, the Jira projects, and the GCP bill.

As at 2026-09-16

Two origination platforms, one vendor core, and the same engineering habits on both sides

2 LOAN ORIGINATION PLATFORMS IN PRODUCTION: BRAVO (JAVA, CAMUNDA) AND LORA (GO, TEMPORAL)	1 VENDOR CORE LOAN SYSTEM BEHIND BOTH: CONFINS, WHOSE SOURCE WE DO NOT HOLD	~122 PRODUCTION SERVICES ON OUR GKE ESTATE, IN GO, JAVA AND NODE.JS	Rp 636M A MONTH ON THREE LOGGING PLATFORMS, ABOUT 11× THE TIER THE MIGRATION WOULD RETIRE
---	---	---	---

WHAT WE RUN AND WHO RUNS IT

Bravo is the older platform. It carries most of the loan book and most of the money, on a workflow engine whose community support ended in October 2025. Its teams are cut by loan step: scoring and underwriting, survey and verification, operations, contract and collateral. **LORA** is the re-architecture. It runs one product family in production, on Temporal Cloud, with three cross-domain teams that take a loan end to end. **CONFINS** is the vendor servicing core both platforms hand a loan to at go-live.

WHAT IS WRONG, IN ONE PARAGRAPH

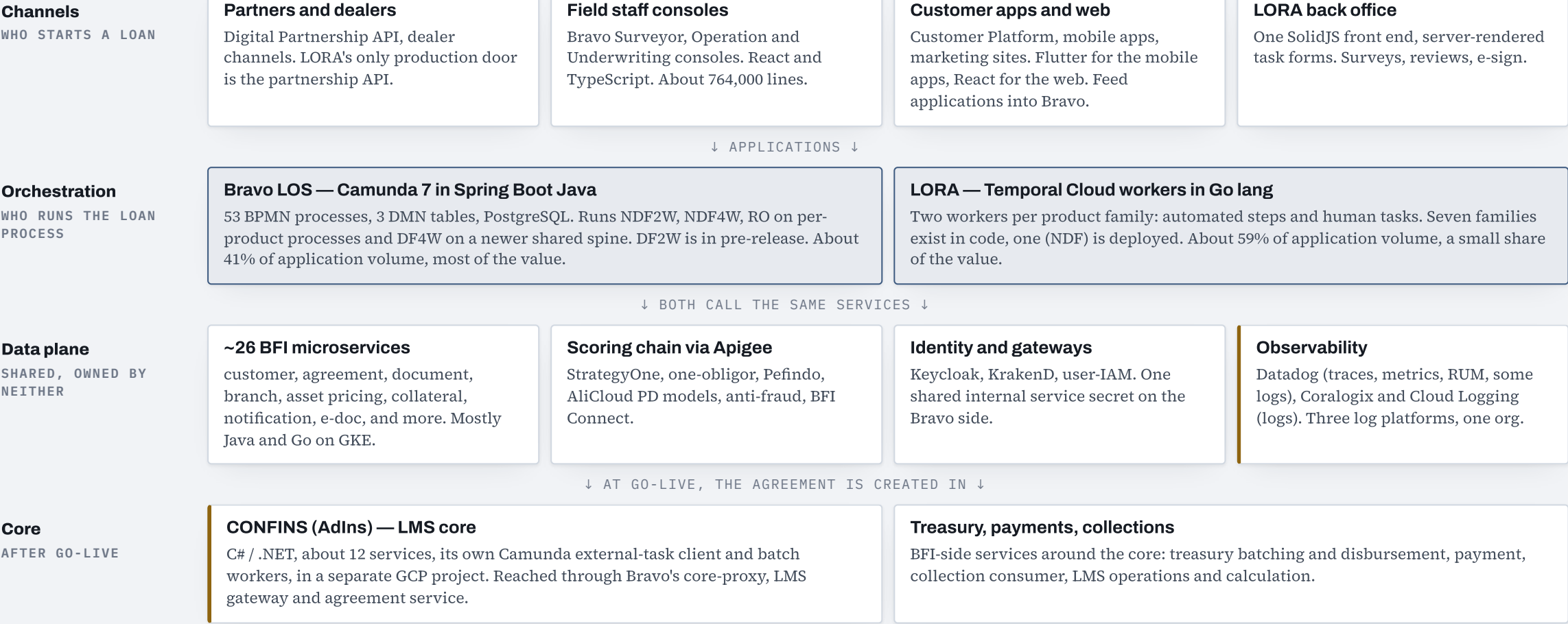
Neither platform tests its orchestration layer. Both built a journey test suite and left it unable to run. Both watch for the wrong failure mode, so the failures they actually have never alert. Logging replaced tracing as the way engineers look at production, and it now costs more than the platform it observes. A security incident in the Bravo engine went unnoticed for three months. And the analysis that found all this was itself wrong several times and had to be corrected, which is a finding about how we measure.

The stacks differ. The habits do not. That is why we are asking a way-of-working consultant, not a platform vendor.

Terms you will hear us use

TERM	WHAT IT MEANS HERE
LOS	Loan Origination System. Everything from a customer's first application to the loan going live. Bravo and LORA are both an LOS.
LMS / CONFINS	Loan Management System, the servicing core that holds agreements, instalments and payments after go-live. Ours is CONFINS, a vendor product from AdIns.
Bravo	The current LOS. Java, Spring Boot, an embedded Camunda BPMN engine, PostgreSQL, three React operator consoles.
LORA	Loan Origination Re-Architected. The replacement LOS. Go, Temporal Cloud workflows, ArangoDB document store, JSON Schema everywhere.
BPMN / GSM	Two ways to model a process. BPMN is a flowchart the engine executes. Guard-Stage-Milestone (GSM) is declarative: a step runs when its data preconditions are met. Bravo is BPMN, LORA is a GSM and Temporal hybrid.
NDF, DF, 2W, 4W, RO	Product families. NDF is non-dealer financing (the partnership door), DF is dealer financing, 2W and 4W are two- and four-wheel vehicles, RO is repeat order. So NDF2W is a two-wheel loan through a partner.
Data plane	About 26 shared BFI microservices (customer, agreement, document, scoring, and so on) that both platforms call. Neither platform owns them.
Squad	A BFI delivery team. Bravo squads are named for a loan step. LORA is one squad with three sub-teams.
Archetype 1 / 2	Our two team shapes. Archetype 1: teams split by domain of the loan steps (Bravo). Archetype 2: cross-domain teams that take a loan end to end (LORA 1, 2, 3).
Datadog, Coralogix, Cloud Logging	The three places our logs go today. Datadog also holds traces, metrics and browser telemetry (RUM). We are standardising on Datadog.

Two front doors, one shared data plane, one vendor core



Bravo: a Java monolith-of-processes with three React consoles around it

LAYER	WHAT IT IS	WHAT WE MEASURED
Runtime	Java 17, Spring Boot 3.5.16. One service, <code>bravo-bpm-service</code> , plus a separate Sharia deployment of the same code.	Boot 3.5 open-source support ended 30 Jun 2026. Java 17 premier support ends 30 Sep 2026.
Workflow engine	Camunda 7 Community Edition 7.23, embedded with its web app and REST API. 53 BPMN processes, 3 DMN decision tables, 276 Java delegates.	Community line closed 14 Oct 2025. No patches, no licence held. A fork swap plus Boot 4 is 18-33 engineer-days.
Data	PostgreSQL on Cloud SQL. 238 JPA entities. Status history written by a database trigger. 1,350 Flyway migrations. Camunda history kept 90 days.	The durable substrate is the database, not the logs. Good for reconstruction, poor for alerting.
Integration	113 typed Feign clients, one per upstream. RabbitMQ. Keycloak plus one shared internal secret.	Per-upstream error attribution is structural. Test doubles are one static shared mock; nothing can make an upstream fail on demand.
Front end	Three React and TypeScript consoles: Surveyor, Operation, Underwriting. 763,861 lines, 829 test files, Vitest. Unit tests block every merge.	Shared design system drifted to three versions ; React 17 on two, 18 on one; Node 18 and 20; Vite 4 and 7.
Journey tests	Cypress with Cucumber. 595 feature files, 28,268 steps, 46 authors.	Frozen 21 Nov 2023. The one scheduled workflow cannot fail.
Products live	NDF2W, NDF4W and R0 on per-product processes. DF4W on the newer shared spine. DF2W in UAT with a penetration test running.	118,253 applications in August 2026. The shared spine carries 5.5% of volume.
Delivery	Jira <code>BLCS</code> (Scoring & Underwriting) and <code>LN</code> (Surveyor & Verificator). Four repositories. 25 people committed in the last 12 months.	Median cycle time 1-2 days in August. Orchestration tier costs about Rp 58M a month.

LORA: schema-driven Go workers on Temporal, one document per loan

LAYER	WHAT IT IS	WHAT WE MEASURED
Runtime	Go. 13 repositories. A shared <code>lora-process-sdk</code> gives every worker its Temporal, document and schema plumbing.	Four SDK versions live in the fleet , about 60 duplicated activity packages, one 883-line forked task executor. The SDK repo has no CI.
Workflow engine	Temporal Cloud. Guard-Stage-Milestone hybrid: an activity runs when its data preconditions are met (155 precondition sites). Human tasks are hand-written state machines.	About 12,000 lines of state machine inside the declarative shell. Retry is unbounded: no maximum attempts, no non-retryable error types.
Data	One JSON document per loan, held in workflow memory, checkpointed to ArangoDB. 500+ JSON Schemas in a schema service; Go field constants are generated (about 1,100 for NDF).	Every business activity costs three Temporal actions. Bookkeeping is 63.5% of activity executions. Temporal is 32% of LORA's bill.
Integration	A gateway with about 290 inner proxies to BFI services and external proxies to partners, all validated against schema. NATS JetStream and RabbitMQ.	All 290 proxies share one route , so 21,000 gateway 500s a week cannot be attributed to an upstream in APM. LORA calls 26 Bravo data-plane services and never Bravo's engine.
Front end	One back-office app in SolidJS. Task forms are server-rendered HTML (about 300 templates) hydrated by web components.	416,000 browser errors over 147,000 views in 7 days. About 17,600 content-security-policy violations block the live task stream.
Tests	608 Go test files. QA holds a Playwright suite with nightly runs and <code>lora-super-test</code> , 150 scenario tests with a Go mock interceptor that can fail any upstream on demand.	Four repositories have zero tests. The planner's scheduling logic has none. The nightly has failed the same 199 API cases every night; 59 of 83 web cases have never passed.
Products live	Seven families in code, each with its own document, worker pair and queue. One is deployed: NDF (90% two-wheel, 10% four-wheel).	Three of the six others have no production deploy workflow. No feature flag gates a family.
Delivery	Jira <code>BL</code> . One squad, three sub-teams (LORA 1, 2, 3), end-to-end task execution.	Median cycle time 2 days in August. All-in platform cost about Rp 431M a month, flat with volume.

CONFINS: the core we depend on, cannot read, and did not know we were paying to log

Both platforms hand a loan to CONFINS at go-live. It is a vendor product, it runs in its own GCP project, and it is the largest single source of production log volume we pay for.

WHAT IT IS

AdIns CONFINS, written in C# on .NET. About twelve services: accounts receivable, foundation, LOS, amendment, payment, cash and bank, AR maintenance, government regulation, reconciliation, recovery, MoU. Plus its own Camunda external-task client and a family of batch workers. It runs in `core-system-prod`, a separate project and cluster from the Bravo estate. Bravo reaches it through a core proxy, an LMS gateway and the agreement service. The agreement service alone emits 7.4 million spans a week and **sends zero logs to Datadog.**

WHAT IS WRONG WITH HOW WE RUN IT

52 of our 122 production services have no source code we can see, and CONFINS and Treasury are most of them. The service catalog owner field is empty. The deployment repository for it is not reachable to the people doing this analysis. Its logger writes every request and response body to disk: agreement numbers, customer numbers, outstanding balances, amortisation schedules. And because the pods write to a shared host path, **the Datadog Agent re-reads a month of dead pods' files at every rollout.** That one service was about a third of all indexed production log events. A first analysis called those lines "blank" because nobody opened the attributes.

~12

CONFINS SERVICES IN PRODUCTION, NONE WITH SOURCE IN OUR REPOSITORIES

~36M

LOG EVENTS A WEEK INGESTED FROM THE CONFINS FAMILY, ABOUT 12M INDEXED

~90%

OF THE LARGEST SERVICE'S VOLUME IS RE-INGESTION OF DEAD PODS' FILES, NOT TRAFFIC

1 day

OF SRE WORK TO FIX THE COLLECTION PATH, NEEDING NO VENDOR ACCESS

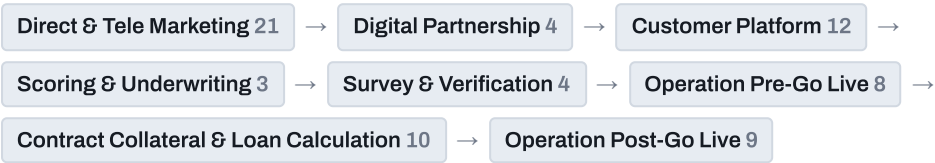
The way-of-working question here is ownership. A system both platforms depend on has no named owner, no readable source, and a logging bill nobody had attributed to it.

Archetype 1 cuts teams by loan step. Archetype 2 cuts them by product, end to end.

ARCHETYPE 1 • BRAVO

Squads split by domain of the loan-application steps

Each squad owns one stage of the journey and the repositories for it. A loan crosses several squads on its way to go-live. The squad folders in our GitHub organisation, with repository counts:



Around them: Insurance 5, Payment 5, Agency 8, Asset Management 8, Supplier Platform 8, Digital Web 5, Customer Mobile Apps 2, and an Internal Service squad with 26 repositories. **Two Jira projects carry Bravo LOS delivery:** **BLCS** for Scoring & Underwriting and **LN** for Surveyor & Verificator. They share a sprint train and have zero cross-project issue links.

ARCHETYPE 2 • LORA

Cross-domain teams that take a loan end to end

LORA is one squad organised as **three sub-teams: LORA 1, LORA 2 and LORA 3**. They work different features in parallel. Most tasks are executed end to end by one team, so members share a baseline understanding of the whole system. This is a team statement and the VMP LORA plan, not something we measured.

- **Why LORA went this way.** A loan is one document with one planner. A "return to survey" change touches survey code and underwriting status writes at once. The architecture has no seam where a domain split would want one.
- **The stated condition for leaving a domain team.** The departing owner first encodes their domain rules as business tests that guard the domain. Tests are the handover.
- **The natural boundary is the product family,** not the loan stage. Seven families, each with its own document, workers and queue.
- **Supporting squads.** LORA Core owns 8 shared repositories; LORA RL owns the revolving-loan family.

Neither shape is a property of its platform. Both are prior organisational decisions, and both can be changed.

What we can actually see each archetype doing

ARCHETYPE 1 • OBSERVED

- **One change touches one or two repositories.** That is the real content of "developing in Bravo is easier". Ticket counts do not support a speed claim in either direction.
- **A bus factor of one.** In two 50-ticket samples, 18 of 20 long-lived Stories were assigned to a single account, on the service carrying most of origination. Sub-tasks spread; the shape of a change does not.
- **Cross-domain work becomes a hand-off.** Underwriting return that must sync surveyor status appears as a ticket in one project with no link to the other.
- **Three consoles, three toolchains.** The same design system at three versions, React 17 and 18, Node 18 and 20. A cutover gate that "runs the console suites" is three different jobs.
- **Backlogs are bulk-loaded.** 45 of 50 tickets in one sample were created in a 37-minute window. Any created-to-resolved metric over that window measures a programme, not a cadence.

ARCHETYPE 2 • OBSERVED

- **One change touches about five repositories.** Schema, SDK, worker, gateway, front end. Work is spread across 22 distinct assignees in 100 sampled tickets.
- **The team's own complaint: "one team must master all domains."** The complaint is confirmed as the organisational cost of one document and one planner. The fix chosen is the end-to-end shape, not merging teams.
- **Nobody owns the platform yet.** Four SDK versions, about 60 duplicated activity packages and a forked executor are the visible cost of every team being stream-aligned. The plan implies a platform team and does not name one.
- **The handover tests do not exist yet.** 62% of automated activity packages have a test, the planner's readiness logic has none, four repositories have zero. You cannot exit into a net that is not built.
- **The top production incident spans both old domains.** 287 survey resets from a survey and task-manager race. A domain-split team structurally could not fix its own top incident.

WHERE THE ANALYSIS ALREADY LANDED

The LORA pack recommends Archetype 2 with a strict order: build the test net first, write the guard packs in pairs with product sign-off, name a platform team, pilot one product family, then roll out. It also names the early-warning signal: **guard tests weakened without sign-off must stay at zero**. This organisation has already silenced an inconvenient alert once rather than fix it, so that rule has to exist before the tests do.

Testing: two large suites that cannot fail, and no test that runs a routing decision

CONVERGES ON BOTH Neither platform tests its orchestration layer Bravo has 2,286 test files across four repositories. Four of them run a process. Its routing lives in BPMN expression strings, a database selector and a JSON matrix that no unit test can reach. LORA has 608 test files; its planner's scheduling logic, the thing that decides what runs, has no test. Neither CI can say which product a change hits.	CONVERGES ON BOTH Both built a journey suite and left it unable to run Bravo: 595 Cypress features, 46 authors, four workflows, frozen November 2023, the one scheduled job written so every step passes. LORA: <code>lora-super-test</code> , 150 scenario tests with real assertions, and a CI workflow with nowhere to run. The nightly Playwright run fails the same 199 API cases every night and the QA dashboard counts them as covered. Neither team knew the other had done the same thing.	CONVERGES ON BOTH Gates that measure and cannot fail Console coverage thresholds are 0%, 0% and 1%. Every SonarQube scan runs without waiting for its quality gate. A Codacy token is missing, so coverage upload fails on every PR. Neither platform sends CI events to Datadog, so nobody can say whether builds are getting slower or redder. The strongest gate in the estate is the consoles' unit tests, which do block a merge.
BRAVO SPECIFIC Bravo cannot make an upstream fail on demand: its mock is one static, shared service. So 50 error handlers, 70 error definitions and 190 escalation paths have never been executed by a test, including a credit decision that bypasses anti-fraud after three failures. On the other side, its 113 typed clients mean the compiler catches stub drift for free, where LORA needs a script against a live schema registry.	THE CHEAP PART The Bravo pack lays out a nine-layer ladder, L0 to L8, ordered by cost. Six layers do not exist. The two that would catch the most, model lint and gateway-expression evaluation, run in milliseconds and are about seven days of work. Five product-matrix journeys would cover the happy path of 100% of production volume; today zero do. This is a decision nobody made, not a mountain.	

Logging: Rp 636M a month to store what a trace would have answered

Rp 636M A MONTH: CLOUD LOGGING 272, CORALOGIX 253, DATADOG 111. A THIRD OF CLOUD LOGGING IS NON-PRODUCTION.	93.6% OF PRODUCTION LOG LINES CANNOT BE PARSED. LINES OVER 16 KB ARE SPLIT BY THE CONTAINER RUNTIME.	~150× THE CONTRACTED LOG INGESTION. 256 GB A MONTH WAS BOUGHT; ABOUT 1.3 TB A DAY IS SENT.	0 LOG LINES ANYWHERE IN PRODUCTION THAT CAN BE JOINED TO A TRACE.
---	--	--	---

Why teams log bodies THE COMPLAINT IS CORRECT	Squads log full request and response bodies because Datadog cannot show them. They are right: no tracer setting puts an HTTP body on a span. The answer, Live Debugger, is blocked because Remote Configuration is failing on 13 production services with about 91,000 failed polls a week. So the workaround was rational, and asking teams to stop without a replacement will be resisted, correctly.
The wrappers made it the default FIX ONCE, NOT PER SERVICE	The shared Java logging library turns body logging on by default and masks six keys by exact match. The Go wrapper's masking is a per-service deployment setting, and in production eight services set it to nothing. One Bravo service alone writes 487,000 bodies a week, 44% of its volume, unmasked.
What ended up in the logs SECURITY, NOT COST	Working chat webhook credentials in the URL of every message sent. A live API secret. Whole HR records about 900,000 times a week: religion, marital status, bank account, personal mobile. Customer phone numbers on every "not found" response. Each has an owner outside the logging programme and a clock on it.
Coverage gaps and split names TELEMETRY PLUMBING	Eleven busy services send no logs to Datadog at all. Eight are split across two service names, one for logs and one for traces, so they can never join. A family of monitors queries service names that do not exist and reports healthy forever. Six services emit neither logs nor traces.

The sequencing is the point. Clean the stream first, then enable better ingestion on a stream half the size. Enabling it today moves the waste from one invoice to another.

Five habits that show up on both platforms and in the vendor estate

HABIT 1 Signals get silenced, not fixed Bravo excluded its most informative engine error from its one business monitor. LORA excluded the go-live message from its activity-error alert and left loans wedged at attempt 26,707. The journey suite's steps all end in "pass anyway". Health monitors filter to 5xx only. In every case the signal was inconvenient and the fix was to stop looking.	HABIT 2 Instruments are on, nobody reads them All four Bravo consoles and both LORA front ends had browser monitoring switched on for months. 2.4 million and 416,000 errors a week, respectively. Nobody had opened the dashboards. Camunda's engine logs carry activity names and versions; nobody ranked failures by them. LORA's APM spans carry the loan id; nobody grouped stuck loans by them.	HABIT 3 "Absence of one failure" means healthy Bravo treats "not 5xx" as healthy and fails by 4xx and by parking a process. LORA treats "still running" as healthy and half its loans never finish. Both platforms' one business monitor is blind to the failure mode each actually has.
HABIT 4 Shared things have no owner 52 production services with no readable source. Feature flags in five mechanisms with no registry. A design system at three versions. Four SDK versions. Three log platforms and nobody can say what the second one is for. A catalog owner field that is empty for the core system. Every duplication we found sits where ownership stops.	HABIT 5 Gates that measure but cannot fail Coverage at 0% thresholds. Quality gates never awaited. Scheduled jobs that cannot go red. A nightly that has failed the same cases for six weeks and counts as coverage. The estate is well instrumented for looking good and poorly instrumented for stopping a merge.	

These are ways of working, not stack choices. Every one of them would survive a platform migration intact.

Reading list, in the order we would suggest

DOCUMENT	WHAT IT SETTLES
<code>bravo-analysis/docs/compare.md</code>	The synthesis. Bravo and LORA side by side on production findings, the eight convergences, the guardrails, Decision A and Decision B.
<code>lora-workspace/docs/production-findings/README.md</code>	The LORA pack's front door. Every production complaint, its verdict, and the document that answers it.
<code>production-findings/Current LORA challenges in Production.md</code>	The LORA team's own problem statement, in their words. Quoted verbatim everywhere else; do not paraphrase it.
<code>bravo-people.md</code> · <code>lora people.md</code>	Team shape, throughput, cycle time, bus factor, onboarding, and the Archetype 2 plan with its challenges and sequencing.
<code>bravo-testing.md</code> · <code>lora testing.md</code> , <code>testing-gates.md</code>	Test inventories, what CI actually gates, the L0-L8 ladder, the QA strategy, the six merge gates that would have caught the incident.
<code>bravo-observability.md</code> · <code>lora observability.md</code> , <code>reliability.md</code>	What each platform can see, the monitors that cannot fire, infinite retry, wedged loans, the intervention rate.
<code>bravo-analysis/docs/logging/</code>	80 files. Start with README.md and logging-cost.md. One file per service with what it logs, what its production manifest says, and the open pull request.
<code>compare-architecture.md</code>	BPMN versus Guard-Stage-Milestone, claim by claim, against the code. The §7 verdict on how much the paradigm actually decided.
<code>option-summary.md</code> , <code>option-1/2/3.md</code>	Bravo's three runtime options: engine fork, Temporal port, LORA migration, with effort and cost.
<code>decks/ (Bravo)</code> · <code>production-findings/*.html (LORA)</code>	Eleven Bravo-side decks, this one included, and seven LORA-side decks, each with a PDF, for specific audiences: CTO, QA, developers, Datadog account team.

Every document separates confirmed from refuted claims, cites file and line, and marks its own corrections in place. Please read the corrections. They are where the lessons are.